

# **SVX Library Class Catalog**

SystemVerilog Extension Library — Core Library (**src/**)

Version 2.0.1 beta

Mark Glasser

September 2026

# Contents

|          |                                         |           |
|----------|-----------------------------------------|-----------|
| <b>1</b> | <b>Types</b>                            | <b>3</b>  |
|          | svx_types.svh . . . . .                 | 3         |
| 1.1      | type_handle_base . . . . .              | 3         |
| 1.2      | type_handle #(T) . . . . .              | 3         |
| 1.3      | type_container_base . . . . .           | 3         |
| 1.4      | type_container #(T) . . . . .           | 4         |
| 1.5      | type_match #(T1, T2) . . . . .          | 4         |
| 1.6      | typeid #(T) . . . . .                   | 5         |
| <b>2</b> | <b>Traits</b>                           | <b>6</b>  |
| 2.1      | void_traits . . . . .                   | 6         |
| 2.2      | object_traits . . . . .                 | 6         |
| 2.3      | class_traits #(T) . . . . .             | 6         |
| 2.4      | base_int_traits #(T) . . . . .          | 7         |
| 2.5      | base_unsigned_int_traits #(T) . . . . . | 7         |
| 2.6      | base_signed_int_traits #(T) . . . . .   | 7         |
| 2.7      | base_four_state_traits #(T) . . . . .   | 7         |
| 2.8      | integer_traits . . . . .                | 8         |
| 2.9      | reg_traits . . . . .                    | 8         |
| 2.10     | bit_vector_traits #(N) . . . . .        | 8         |
| 2.11     | real_base_traits #(T) . . . . .         | 8         |
| 2.12     | string_traits . . . . .                 | 8         |
| <b>3</b> | <b>Containers</b>                       | <b>10</b> |
| 3.1      | void_t . . . . .                        | 10        |
| 3.2      | object . . . . .                        | 10        |
| 3.3      | container . . . . .                     | 10        |
| 3.4      | typed_container #(T, P) . . . . .       | 11        |
| 3.5      | vector #(T, P) . . . . .                | 11        |
| 3.6      | map #(KEY, T, P) . . . . .              | 12        |
| 3.7      | singleton_map #(KEY, T, P) . . . . .    | 13        |
| 3.8      | queue #(T, P) . . . . .                 | 13        |
| 3.9      | fixed_size_queue #(T, P) . . . . .      | 13        |
| 3.10     | stack #(T, P) . . . . .                 | 14        |
| 3.11     | deque #(T, P) . . . . .                 | 14        |
| 3.12     | circ_buf #(T, P, S) . . . . .           | 15        |
| 3.13     | pair #(T1, T2) . . . . .                | 15        |
| 3.14     | triple #(T1, T2, T3) . . . . .          | 16        |
| 3.15     | quadruple #(T1, T2, T3, T4) . . . . .   | 16        |
| 3.16     | sorter #(T, P) . . . . .                | 16        |

|          |                                            |           |
|----------|--------------------------------------------|-----------|
| <b>4</b> | <b>Iterators</b>                           | <b>18</b> |
| 4.1      | iterator_intf_base #(T, P) . . . . .       | 18        |
| 4.2      | fwd_intf #(T, P) . . . . .                 | 18        |
| 4.3      | bkwd_intf #(T, P) . . . . .                | 19        |
| 4.4      | random_intf #(T, P) . . . . .              | 19        |
| 4.5      | list_iterator_base #(T, P) . . . . .       | 19        |
| 4.6      | list_random_iterator #(T, P) . . . . .     | 20        |
| 4.7      | list_iterator #(T, P) . . . . .            | 20        |
| 4.8      | map_iterator_base #(KEY, T, P) . . . . .   | 21        |
| 4.9      | map_random_iterator #(KEY, T, P) . . . . . | 21        |
| 4.10     | map_iterator #(KEY, T, P) . . . . .        | 22        |
| 4.11     | permute_iterator_base #(T, P) . . . . .    | 23        |
| 4.12     | permute_random_iterator #(T, P) . . . . .  | 23        |
| 4.13     | permute_iterator #(T, P) . . . . .         | 24        |
| 4.14     | range_base . . . . .                       | 24        |
| 4.15     | range #(T, P) . . . . .                    | 25        |
| 4.16     | map_range #(KEY, T, P) . . . . .           | 25        |
| <b>5</b> | <b>Algorithms</b>                          | <b>27</b> |
| 5.1      | algo #(T, P) . . . . .                     | 27        |
| 5.2      | accum #(T, P, A) . . . . .                 | 27        |
| 5.3      | fcn_base . . . . .                         | 28        |
| 5.4      | fcn #(T) . . . . .                         | 28        |
| 5.5      | accum_fcn #(T, A) . . . . .                | 28        |
| 5.6      | predicate_base . . . . .                   | 28        |
| 5.7      | predicate #(T) . . . . .                   | 28        |
| 5.8      | and_pred #(T) . . . . .                    | 29        |
| 5.9      | or_pred #(T) . . . . .                     | 29        |
| 5.10     | not_pred #(T) . . . . .                    | 29        |
| 5.11     | always_true #(T) . . . . .                 | 29        |
| 5.12     | always_false #(T) . . . . .                | 30        |
| <b>6</b> | <b>Linked Structures</b>                   | <b>31</b> |
| 6.1      | node . . . . .                             | 31        |
| 6.2      | tree . . . . .                             | 31        |
| 6.3      | tree_iterator . . . . .                    | 32        |

# Types

## `svx_types.svh`

This file defines primitive type aliases (`int8_t`–`uint128_t`, `size_t`, `index_t`, `signed_index_t`, `pid_t`) and the boolean constants `true` and `false`. It contains no classes.

### 1.1 `type_handle_base`

**Kind:** virtual class

**Extends:** `object`

Abstract base providing a virtual type-handle accessor

| Method                       | Returns                       | Arguments | Description                                           |
|------------------------------|-------------------------------|-----------|-------------------------------------------------------|
| <code>get_type_handle</code> | <code>type_handle_base</code> | —         | Return the type handle for this object (pure virtual) |

### 1.2 `type_handle #(T)`

**Kind:** class

**Extends:** `type_handle_base`

Singleton type token; one instance per unique type T

| Method                       | Returns                       | Arguments | Description                                                 |
|------------------------------|-------------------------------|-----------|-------------------------------------------------------------|
| <code>get_type</code>        | <code>this_t</code>           | —         | Return (or create) the singleton type handle for T (static) |
| <code>get_type_handle</code> | <code>type_handle_base</code> | —         | Return this singleton as a base-class handle                |

### 1.3 `type_container_base`

**Kind:** class

Non-parameterized base for typed containers; provides `convert2string` and handle access

| Method                       | Returns                       | Arguments | Description                                          |
|------------------------------|-------------------------------|-----------|------------------------------------------------------|
| <code>convert2string</code>  | <code>string</code>           | —         | Return a human-readable string for the stored object |
| <code>get_type_handle</code> | <code>type_handle_base</code> | —         | Return the type handle (returns null in base)        |

## 1.4 `type_container #(T)`

**Kind:** class

**Extends:** `type_container_base`

Wraps a single object of type T together with its type handle

**Constructor:** `new()`

| Method                       | Returns                       | Arguments      | Description                                   |
|------------------------------|-------------------------------|----------------|-----------------------------------------------|
| <code>get_type_handle</code> | <code>type_handle_base</code> | —              | Return the type handle for type T             |
| <code>set</code>             | <code>void</code>             | <code>x</code> | Store a new object, replacing the current one |
| <code>get</code>             | <code>T</code>                | —              | Retrieve the stored object                    |

## 1.5 `type_match #(T1, T2)`

**Kind:** class

Static utility to test whether two type parameters match or one derives from the other

| Method                            | Returns           | Arguments               | Description                                                    |
|-----------------------------------|-------------------|-------------------------|----------------------------------------------------------------|
| <code>is_match</code>             | <code>bit</code>  | <code>line, file</code> | Assert that T1 and T2 are the same type; fatal if not (static) |
| <code>test_is_match</code>        | <code>bit</code>  | —                       | Return 1 if T1 and T2 are the same type (static)               |
| <code>fail_match</code>           | <code>void</code> | <code>line, file</code> | Emit a fatal message that T1 and T2 do not match (static)      |
| <code>is_derived_from</code>      | <code>bit</code>  | <code>line, file</code> | Assert T1 is derived from T2; fatal if not (static)            |
| <code>test_is_derived_from</code> | <code>bit</code>  | —                       | Return 1 if T1 is derived from T2 (static)                     |
| <code>fail_derived</code>         | <code>void</code> | <code>line, file</code> | Emit a fatal message that T1 is not derived from T2 (static)   |

## 1.6 typeid #(T)

**Kind:** class

Static utility to classify a type parameter (integral, two-state, four-state, real, string)

| Method                          | Returns | Arguments | Description                                                         |
|---------------------------------|---------|-----------|---------------------------------------------------------------------|
| <code>is_int</code>             | bit     | —         | Return 1 if T is any integral type (static)                         |
| <code>is_int_fail</code>        | bit     | —         | Return 1 if T is integral; print error and finish if not (static)   |
| <code>is_two_state</code>       | bit     | —         | Return 1 if T is a two-state integral type (static)                 |
| <code>is_two_state_fail</code>  | bit     | —         | Return 1 if T is two-state; print error and finish if not (static)  |
| <code>is_four_state</code>      | bit     | —         | Return 1 if T is a four-state integral type (static)                |
| <code>is_four_state_fail</code> | bit     | —         | Return 1 if T is four-state; print error and finish if not (static) |
| <code>is_string</code>          | bit     | —         | Return 1 if T is string (static)                                    |
| <code>is_string_fail</code>     | bit     | —         | Return 1 if T is string; print error and finish if not (static)     |
| <code>is_real</code>            | bit     | —         | Return 1 if T is real (static)                                      |
| <code>is_real_fail</code>       | bit     | —         | Return 1 if T is real; print error and finish if not (static)       |

# Traits

## 2.1 void\_traits

**Kind:** class

**Extends:** void\_t

Root traits class; all traits classes must extend this; provides type flags and utility methods

| Method      | Returns | Arguments                 | Description                                            |
|-------------|---------|---------------------------|--------------------------------------------------------|
| equal       | bit     | a, b                      | Return 1 (void objects are always equivalent) (static) |
| compare     | int32_t | a, b                      | Return int cast of !equal(a,b) (static)                |
| sort        | void    | vec                       | No-op (static)                                         |
| check_trait | bit     | trait, exp,<br>line, file | Assert trait==exp; call fail() if not (static)         |
| fail        | void    | line, file                | Emit a fatal traits-failure message (static)           |

## 2.2 object\_traits

**Kind:** class

**Extends:** void\_traits

Traits for classes derived from object; delegates equal/compare to object::compare()

| Method  | Returns | Arguments | Description                          |
|---------|---------|-----------|--------------------------------------|
| equal   | bit     | a, b      | Return 1 if a.compare(b)==0 (static) |
| compare | int32_t | a, b      | Return a.compare(b) (static)         |
| sort    | void    | vec       | No-op placeholder (static)           |

## 2.3 class\_traits #(T)

**Kind:** class

**Extends:** void\_traits

Traits for arbitrary class types; equality is pointer equality

| Method               | Returns              | Arguments         | Description                                               |
|----------------------|----------------------|-------------------|-----------------------------------------------------------|
| <code>equal</code>   | <code>bit</code>     | <code>a, b</code> | Return 1 if <code>a==b</code> (pointer equality) (static) |
| <code>compare</code> | <code>int32_t</code> | <code>a, b</code> | Return int cast of <code>!equal(a,b)</code> (static)      |
| <code>sort</code>    | <code>void</code>    | <code>vec</code>  | No-op placeholder (static)                                |

## 2.4 `base_int_traits #(T)`

**Kind:** class

**Extends:** `void_traits`

Base traits for all integral types; `equal/compare` use native operators

| Method               | Returns              | Arguments         | Description                                                                       |
|----------------------|----------------------|-------------------|-----------------------------------------------------------------------------------|
| <code>equal</code>   | <code>bit</code>     | <code>a, b</code> | Return 1 if <code>a==b</code> (static)                                            |
| <code>compare</code> | <code>int32_t</code> | <code>a, b</code> | Return 1 if <code>a&gt;b</code> , -1 if <code>a&lt;b</code> , 0 if equal (static) |
| <code>sort</code>    | <code>void</code>    | <code>vec</code>  | Sort queue in-place using built-in sort (static)                                  |

## 2.5 `base_unsigned_int_traits #(T)`

**Kind:** class

**Extends:** `base_int_traits #(T)`

Traits variant marking the type as unsigned (sets `is_unsigned=true`, `is_signed=false`)

*No public methods.*

## 2.6 `base_signed_int_traits #(T)`

**Kind:** class

**Extends:** `base_int_traits #(T)`

Traits variant marking the type as signed (sets `is_signed=true`, `is_unsigned=false`)

*No public methods.*

## 2.7 `base_four_state_traits #(T)`

**Kind:** class

**Extends:** `base_int_traits #(T)`

Traits base for four-state (logic/reg/integer/time) types

*No public methods.*

## 2.8 integer\_traits

**Kind:** class

**Extends:** `base_four_state_traits #(integer)`

Traits specialisation for the four-state signed integer type

*No public methods.*

## 2.9 reg\_traits

**Kind:** class

**Extends:** `base_four_state_traits #(reg)`

Traits specialisation for reg (also aliased as logic\_traits and time\_traits)

*No public methods.*

## 2.10 bit\_vector\_traits #(N)

**Kind:** class

**Extends:** `base_int_traits #(bit[N-1:0])`

Traits for packed bit-vector types of width N

*No public methods.*

## 2.11 real\_base\_traits #(T)

**Kind:** class

**Extends:** `void_traits`

Base traits for floating-point types; equality uses epsilon comparison

| Method               | Returns              | Arguments         | Description                                                        |
|----------------------|----------------------|-------------------|--------------------------------------------------------------------|
| <code>equal</code>   | <code>bit</code>     | <code>a, b</code> | Return 1 if                                                        |
| <code>compare</code> | <code>int32_t</code> | <code>a, b</code> | Return 1 if $a > b$ , -1 if $a < b$ , 0 if within epsilon (static) |
| <code>sort</code>    | <code>void</code>    | <code>vec</code>  | Sort queue using built-in sort with (item) (static)                |

## 2.12 string\_traits

**Kind:** class

**Extends:** `void_traits`

Traits for string type; comparison is lexicographic

| Method               | Returns              | Arguments         | Description                                                                       |
|----------------------|----------------------|-------------------|-----------------------------------------------------------------------------------|
| <code>equal</code>   | <code>bit</code>     | <code>a, b</code> | Return 1 if <code>a==b</code> (static)                                            |
| <code>compare</code> | <code>int32_t</code> | <code>a, b</code> | Return 1 if <code>a&gt;b</code> , -1 if <code>a&lt;b</code> , 0 if equal (static) |
| <code>sort</code>    | <code>void</code>    | <code>vec</code>  | Sort queue lexicographically (static)                                             |

# Containers

## 3.1 void\_t

**Kind:** class

The empty base class; root of the SVX class hierarchy

*No public methods.*

## 3.2 object

**Kind:** class

**Extends:** void\_t

Basic object providing to\_str, compare, and copy virtual interfaces

| Method  | Returns | Arguments | Description                                         |
|---------|---------|-----------|-----------------------------------------------------|
| to_str  | string  | —         | Return a human-readable string representation       |
| compare | int     | obj       | Compare this object with another; return 0 if equal |
| copy    | void    | rhs       | Copy state from another object into this one        |

## 3.3 container

**Kind:** virtual class

**Extends:** void\_t

Abstract base for any container; specifies size, clear, and is\_empty interface

| Method | Returns | Arguments | Description                                                |
|--------|---------|-----------|------------------------------------------------------------|
| size   | size_t  | —         | Return the number of items in the container (pure virtual) |
| clear  | void    | —         | Remove all items from the container (pure virtual)         |

*(continued on next page...)*

(continued...)

| Method                | Returns          | Arguments | Description                                             |
|-----------------------|------------------|-----------|---------------------------------------------------------|
| <code>is_empty</code> | <code>bit</code> | —         | Return 1 if the container holds no items (pure virtual) |

### 3.4 `typed_container #(T, P)`

**Kind:** virtual class

**Extends:** `container`

Parameterized container base; adds element type T and traits class P

**Constructor:** `new()`

| Method                | Returns             | Arguments | Description                       |
|-----------------------|---------------------|-----------|-----------------------------------|
| <code>size</code>     | <code>size_t</code> | —         | Return 0 (override in subclasses) |
| <code>is_empty</code> | <code>bit</code>    | —         | Return 0 (override in subclasses) |
| <code>clear</code>    | <code>void</code>   | —         | No-op (override in subclasses)    |

### 3.5 `vector #(T, P)`

**Kind:** class

**Extends:** `typed_container #(T, P)`

Dynamic array container backed by a SystemVerilog queue; supports indexed access and bulk append

| Method                | Returns             | Arguments           | Description                                                     |
|-----------------------|---------------------|---------------------|-----------------------------------------------------------------|
| <code>create</code>   | <code>this_t</code> | <code>list</code>   | Create a new vector pre-populated with a literal queue (static) |
| <code>size</code>     | <code>size_t</code> | —                   | Return the number of items in the vector                        |
| <code>is_empty</code> | <code>bit</code>    | —                   | Return 1 if the vector contains no elements                     |
| <code>clear</code>    | <code>void</code>   | —                   | Remove all elements, returning the vector to an empty state     |
| <code>extend</code>   | <code>void</code>   | <code>sz</code>     | Extend the vector by sz empty elements                          |
| <code>write</code>    | <code>void</code>   | <code>idx, t</code> | Write element t at index idx, extending if needed               |
| <code>read</code>     | T                   | <code>idx</code>    | Read element at idx; return empty element if out of range       |

(continued on next page...)

(continued...)

| Method  | Returns | Arguments | Description                                                   |
|---------|---------|-----------|---------------------------------------------------------------|
| copy    | void    | vec       | Shallow-copy vec into this vector                             |
| clone   | this_t  | —         | Return a new vector that is a shallow copy of this one        |
| compare | int     | v         | Return 0 if vectors are equal, 1 otherwise                    |
| equal   | bit     | v         | Return 1 if v has the same size and all elements are P::equal |
| sort    | void    | —         | Sort in-place using the P::sort function                      |
| appendv | void    | v         | Append a literal queue to the vector                          |
| appendc | void    | t         | Append a single element to the vector                         |
| append  | void    | v         | Append another vector object to this vector                   |

### 3.6 map #(KEY, T, P)

**Kind:** class

**Extends:** typed\_container #(T, P)

Class-based dynamic associative array; allows sparse allocation and pass-by-reference

| Method   | Returns | Arguments | Description                                               |
|----------|---------|-----------|-----------------------------------------------------------|
| get      | T       | key       | Return the item for key; return P::empty if not found     |
| insert   | bit     | key, item | Insert (key,item); return 1 if key is new, 0 if duplicate |
| size     | size_t  | —         | Return the number of entries in the map                   |
| is_empty | bit     | —         | Return 1 if the map is empty                              |
| delete   | bit     | key       | Remove the entry with key; return 1 on success            |
| clear    | void    | —         | Remove all entries from the map                           |
| exists   | bit     | key       | Return 1 if key exists in the map                         |
| copy     | void    | rhs       | Shallow-copy rhs into this map                            |
| clone    | this_t  | —         | Return a new map that is a shallow copy of this one       |
| compare  | int32_t | m         | Return 0 if maps are equal, 1 otherwise                   |
| equal    | bit     | m         | Return 1 if both maps have same keys and P::equal values  |

(continued on next page...)

(continued...)

| Method | Returns | Arguments | Description                                             |
|--------|---------|-----------|---------------------------------------------------------|
| first  | bit     | index     | Set index to the first (smallest) key; for iterator use |
| last   | bit     | index     | Set index to the last (largest) key; for iterator use   |
| next   | bit     | index     | Advance index to the next key; for iterator use         |
| prev   | bit     | index     | Retreat index to the previous key; for iterator use     |

### 3.7 singleton\_map #(KEY, T, P)

**Kind:** class

**Extends:** map #(KEY, T, P)

A map with exactly one global instance; obtain it via get\_inst()

| Method   | Returns | Arguments | Description                                        |
|----------|---------|-----------|----------------------------------------------------|
| get_inst | this_t  | —         | Return (or create) the singleton instance (static) |

### 3.8 queue #(T, P)

**Kind:** class

**Extends:** vector #(T, P)

FIFO queue; put() inserts at head, get() removes and returns from tail

| Method   | Returns | Arguments | Description                                                      |
|----------|---------|-----------|------------------------------------------------------------------|
| put      | void    | t         | Insert item at the head of the queue                             |
| get      | T       | —         | Remove and return the item at the tail; return P::empty if empty |
| peek     | T       | —         | Return the tail item without removing it                         |
| is_empty | bit     | —         | Return 1 if the queue contains no items                          |
| clone    | this_t  | —         | Return a new queue that is a copy of this one                    |

### 3.9 fixed\_size\_queue #(T, P)

**Kind:** class

**Extends:** queue #(T, P)

Queue with a configurable maximum capacity; put() silently drops items when full

**Constructor:** `new(n)`

| Method                           | Returns             | Arguments      | Description                                                           |
|----------------------------------|---------------------|----------------|-----------------------------------------------------------------------|
| <code>set_max_size</code>        | <code>void</code>   | <code>n</code> | Change the maximum number of entries allowed                          |
| <code>get_max_size</code>        | <code>size_t</code> | —              | Return the current maximum size                                       |
| <code>put</code>                 | <code>void</code>   | <code>t</code> | Insert item if queue is not full; update <code>push_ok</code> flag    |
| <code>last_push_succeeded</code> | <code>bit</code>    | —              | Return 1 if the most recent <code>put()</code> succeeded              |
| <code>is_full</code>             | <code>bit</code>    | —              | Return 1 if the queue is at maximum capacity                          |
| <code>copy</code>                | <code>void</code>   | <code>q</code> | Copy another <code>fixed_size_queue</code> into this one              |
| <code>clone</code>               | <code>this_t</code> | —              | Return a new <code>fixed_size_queue</code> that is a copy of this one |
| <code>equal</code>               | <code>bit</code>    | <code>v</code> | Return 1 if both queues have equal contents and parameters            |

### 3.10 `stack #(T, P)`

**Kind:** class

**Extends:** `vector #(T, P)`

LIFO stack built on vector; push/pop operate on the front of the underlying queue

| Method                | Returns             | Arguments      | Description                                                                    |
|-----------------------|---------------------|----------------|--------------------------------------------------------------------------------|
| <code>pop</code>      | <code>T</code>      | —              | Remove and return the top-of-stack item; return <code>P::empty</code> if empty |
| <code>push</code>     | <code>void</code>   | <code>t</code> | Push item onto the top of the stack                                            |
| <code>peek</code>     | <code>T</code>      | —              | Return the top item without removing it                                        |
| <code>is_empty</code> | <code>bit</code>    | —              | Return 1 if the stack contains no items                                        |
| <code>clone</code>    | <code>this_t</code> | —              | Return a new stack that is a copy of this one                                  |

### 3.11 `deque #(T, P)`

**Kind:** class

**Extends:** `vector #(T, P)`

Double-ended queue supporting push and pop at both ends, plus shuffle and reverse

| Method     | Returns | Arguments | Description                                   |
|------------|---------|-----------|-----------------------------------------------|
| pop_front  | T       | —         | Remove and return the item at the front       |
| pop_back   | T       | —         | Remove and return the item at the back        |
| push_front | void    | t         | Insert item at the front                      |
| push_back  | void    | t         | Insert item at the back                       |
| shuffle    | void    | —         | Randomly reorder all items in-place           |
| reverse    | void    | —         | Reverse the order of all items in-place       |
| clone      | this_t  | —         | Return a new deque that is a copy of this one |

### 3.12 circ\_buf #(T, P, S)

**Kind:** class

**Extends:** typed\_container #(T, P)

Fixed-size circular buffer of capacity S; push\_tail/pop\_head operations

**Constructor:** new()

| Method    | Returns | Arguments | Description                                                |
|-----------|---------|-----------|------------------------------------------------------------|
| size      | size_t  | —         | Return the number of items currently in the buffer         |
| is_empty  | bit     | —         | Return 1 if the buffer contains no items                   |
| clear     | void    | —         | Reset head, tail, and count to zero                        |
| is_full   | bit     | —         | Return 1 if the buffer is at capacity S                    |
| push_tail | void    | t         | Add item at the tail; print error if full                  |
| pop_head  | T       | —         | Remove and return item from the head; print error if empty |

### 3.13 pair #(T1, T2)

**Kind:** class

**Extends:** void\_t

Ordered pair holding two values of potentially different types

**Constructor:** new(a, b)

| Method | Returns | Arguments | Description              |
|--------|---------|-----------|--------------------------|
| first  | T1      | —         | Return the first element |

*(continued on next page...)*

*(continued...)*

| Method                  | Returns           | Arguments      | Description               |
|-------------------------|-------------------|----------------|---------------------------|
| <code>second</code>     | <code>T2</code>   | —              | Return the second element |
| <code>set_first</code>  | <code>void</code> | <code>t</code> | Set the first element     |
| <code>set_second</code> | <code>void</code> | <code>t</code> | Set the second element    |

### 3.14 `triple #(T1, T2, T3)`

**Kind:** class

**Extends:** `pair #(T1, T2)`

Ordered triple; extends pair with a third element

**Constructor:** `new(a, b, c)`

| Method                 | Returns           | Arguments      | Description              |
|------------------------|-------------------|----------------|--------------------------|
| <code>third</code>     | <code>T3</code>   | —              | Return the third element |
| <code>set_third</code> | <code>void</code> | <code>t</code> | Set the third element    |

### 3.15 `quadruple #(T1, T2, T3, T4)`

**Kind:** class

**Extends:** `triple #(T1, T2, T3)`

Ordered quadruple; extends triple with a fourth element

**Constructor:** `new(a, b, c, d)`

| Method                  | Returns           | Arguments      | Description               |
|-------------------------|-------------------|----------------|---------------------------|
| <code>fourth</code>     | <code>T4</code>   | —              | Return the fourth element |
| <code>set_fourth</code> | <code>void</code> | <code>t</code> | Set the fourth element    |

### 3.16 `sorter #(T, P)`

**Kind:** class

Quicksort implementation for queues of type T using P::compare for ordering

| Method            | Returns           | Arguments        | Description                                  |
|-------------------|-------------------|------------------|----------------------------------------------|
| <code>sort</code> | <code>void</code> | <code>vec</code> | Sort the queue in-place (static entry point) |

*(continued on next page...)*

*(continued...)*

| Method | Returns | Arguments | Description                                           |
|--------|---------|-----------|-------------------------------------------------------|
| qsort  | void    | vec, l, r | Recursive quicksort over the slice [l, r]<br>(static) |

# Iterators

## 4.1 `iterator_intf_base #(T, P)`

**Kind:** interface class

Root iterator interface defining `set`, `get`, `skip`, `size`, and `is_empty`

| Method                | Returns             | Arguments             | Description                                                                     |
|-----------------------|---------------------|-----------------------|---------------------------------------------------------------------------------|
| <code>set</code>      | <code>void</code>   | <code>t</code>        | Set the value of the item at the current index (pure virtual)                   |
| <code>get</code>      | <code>T</code>      | —                     | Retrieve the item at the current index (pure virtual)                           |
| <code>skip</code>     | <code>bit</code>    | <code>distance</code> | Move the index forward ( $>0$ ) or backward ( $<0$ ) by distance (pure virtual) |
| <code>size</code>     | <code>size_t</code> | —                     | Return the number of elements in the bound container (pure virtual)             |
| <code>is_empty</code> | <code>bit</code>    | —                     | Return 1 if not bound or the container is empty (pure virtual)                  |

## 4.2 `fwd_intf #(T, P)`

**Kind:** interface class

**Implements:** `iterator_intf_base #(T, P)`

Forward iterator interface; adds `first`, `next`, `is_last`, and `at_end`

| Method               | Returns          | Arguments | Description                                                   |
|----------------------|------------------|-----------|---------------------------------------------------------------|
| <code>first</code>   | <code>bit</code> | —         | Move to the first item; return 1 on success (pure virtual)    |
| <code>next</code>    | <code>bit</code> | —         | Advance to the next item; return 1 on success (pure virtual)  |
| <code>is_last</code> | <code>bit</code> | —         | Return 1 if pointing at the last item (pure virtual)          |
| <code>at_end</code>  | <code>bit</code> | —         | Return 1 if the iterator is past the last item (pure virtual) |

### 4.3 `bkwd_intf #(T, P)`

**Kind:** interface class

**Implements:** `iterator_intf_base #(T, P)`

Backward iterator interface; adds `last`, `prev`, `is_first`, and `at_beginning`

| Method                    | Returns          | Arguments | Description                                                      |
|---------------------------|------------------|-----------|------------------------------------------------------------------|
| <code>last</code>         | <code>bit</code> | —         | Move to the last item; return 1 on success (pure virtual)        |
| <code>prev</code>         | <code>bit</code> | —         | Move to the previous item; return 1 on success (pure virtual)    |
| <code>is_first</code>     | <code>bit</code> | —         | Return 1 if pointing at the first item (pure virtual)            |
| <code>at_beginning</code> | <code>bit</code> | —         | Return 1 if the iterator is before the first item (pure virtual) |

### 4.4 `random_intf #(T, P)`

**Kind:** interface class

**Implements:** `iterator_intf_base #(T, P)`

Random-access iterator interface; adds random selection and seed control

| Method                        | Returns           | Arguments         | Description                                                       |
|-------------------------------|-------------------|-------------------|-------------------------------------------------------------------|
| <code>random</code>           | <code>bit</code>  | —                 | Select a random valid element; return 1 on success (pure virtual) |
| <code>set_seed</code>         | <code>void</code> | <code>seed</code> | Set the RNG seed (pure virtual)                                   |
| <code>set_default_seed</code> | <code>void</code> | —                 | Reset to the default RNG seed (pure virtual)                      |

### 4.5 `list_iterator_base #(T, P)`

**Kind:** virtual class

Abstract base for iterators that traverse a vector (or vector-derived) container

**Constructor:** `new(list)`

| Method                 | Returns           | Arguments         | Description                                           |
|------------------------|-------------------|-------------------|-------------------------------------------------------|
| <code>bind_list</code> | <code>void</code> | <code>list</code> | Bind (or rebind) a vector to the iterator             |
| <code>set</code>       | <code>void</code> | <code>t</code>    | Write <code>t</code> to the item at the current index |
| <code>get</code>       | <code>T</code>    | —                 | Read the item at the current index                    |

*(continued on next page...)*

(continued...)

| Method   | Returns | Arguments | Description                                         |
|----------|---------|-----------|-----------------------------------------------------|
| skip     | bit     | distance  | Move the index by distance (clamped to valid range) |
| size     | size_t  | —         | Return the size of the bound container              |
| is_empty | bit     | —         | Return 1 if unbound or the container is empty       |

## 4.6 list\_random\_iterator #(T, P)

**Kind:** class

**Extends:** list\_iterator\_base #(T, P)

**Implements:** random\_intf #(T, P)

Randomly selects an item in a bound vector

**Constructor:** new(list\_inst)

| Method           | Returns | Arguments | Description                               |
|------------------|---------|-----------|-------------------------------------------|
| set              | void    | t         | Delegate to base-class set                |
| get              | T       | —         | Delegate to base-class get                |
| size             | size_t  | —         | Delegate to base-class size               |
| is_empty         | bit     | —         | Delegate to base-class is_empty           |
| set_seed         | void    | seed      | Seed the RNG                              |
| set_default_seed | void    | —         | Reset RNG to the default seed             |
| random           | bit     | —         | Choose a random index in the bound vector |
| skip             | bit     | distance  | Delegate to base-class skip               |

## 4.7 list\_iterator #(T, P)

**Kind:** class

**Extends:** list\_iterator\_base #(T, P)

**Implements:** fwd\_intf #(T, P), bkwd\_intf #(T, P)

Bidirectional iterator over a vector; supports forward and backward traversal

**Constructor:** new(list\_inst)

| Method | Returns | Arguments | Description                |
|--------|---------|-----------|----------------------------|
| set    | void    | t         | Delegate to base-class set |
| get    | T       | —         | Delegate to base-class get |

(continued on next page...)

(continued...)

| Method       | Returns | Arguments | Description                                         |
|--------------|---------|-----------|-----------------------------------------------------|
| size         | size_t  | —         | Delegate to base-class size                         |
| is_empty     | bit     | —         | Delegate to base-class is_empty                     |
| first        | bit     | —         | Move to index 0; return 1 if container is non-empty |
| next         | bit     | —         | Advance index by 1; return 1 on success             |
| is_last      | bit     | —         | Return 1 if at the last valid index                 |
| at_end       | bit     | —         | Return 1 if index is past the last element          |
| last         | bit     | —         | Move to the last valid index                        |
| prev         | bit     | —         | Retreat index by 1; return 1 on success             |
| is_first     | bit     | —         | Return 1 if at index 0                              |
| at_beginning | bit     | —         | Return 1 if index is before element 0               |
| skip         | bit     | distance  | Delegate to base-class skip                         |

## 4.8 map\_iterator\_base #(KEY, T, P)

**Kind:** virtual class

Abstract base for iterators that traverse a map container

**Constructor:** new(map\_inst)

| Method    | Returns | Arguments | Description                                    |
|-----------|---------|-----------|------------------------------------------------|
| bind_map  | void    | m         | Bind (or rebind) a map to the iterator         |
| get_index | KEY     | —         | Return the key (index) at the current position |
| size      | size_t  | —         | Return the number of entries in the bound map  |
| is_empty  | bit     | —         | Return 1 if unbound or the map is empty        |

## 4.9 map\_random\_iterator #(KEY, T, P)

**Kind:** class

**Extends:** map\_iterator\_base #(KEY, T, P)

**Implements:** random\_intf #(T, P)

Randomly selects an entry in a bound map

**Constructor:** new(map\_inst)

| Method           | Returns | Arguments | Description                             |
|------------------|---------|-----------|-----------------------------------------|
| set              | void    | t         | Insert t at the current map key         |
| get              | T       | —         | Return the value at the current map key |
| set_seed         | void    | seed      | Seed the RNG                            |
| set_default_seed | void    | —         | Reset RNG to the default seed           |
| random           | bit     | —         | Choose a random entry from the map      |
| skip             | bit     | distance  | No-op placeholder (not implemented)     |
| size             | size_t  | —         | Delegate to base-class size             |
| is_empty         | bit     | —         | Delegate to base-class is_empty         |

## 4.10 map\_iterator #(KEY, T, P)

**Kind:** class

**Extends:** map\_iterator\_base #(KEY, T, P)

**Implements:** fwd\_intf #(T, P), bkwd\_intf #(T, P)

Bidirectional iterator over a map; traverses in key order

**Constructor:** new(map\_inst)

| Method       | Returns | Arguments | Description                                      |
|--------------|---------|-----------|--------------------------------------------------|
| set          | void    | t         | Insert t at the current map key                  |
| get          | T       | —         | Return the value at the current map key          |
| get_index    | KEY     | —         | Return the current key                           |
| first        | bit     | —         | Move to the smallest key; return 1 on success    |
| next         | bit     | —         | Advance to the next key; return 1 on success     |
| is_last      | bit     | —         | Return 1 if at the largest key                   |
| at_end       | bit     | —         | Return 1 if past the last key                    |
| last         | bit     | —         | Move to the largest key; return 1 on success     |
| prev         | bit     | —         | Retreat to the previous key; return 1 on success |
| is_first     | bit     | —         | Return 1 if at the smallest key                  |
| at_beginning | bit     | —         | Return 1 if before the first key                 |
| skip         | bit     | distance  | Move forward or backward by distance steps       |

*(continued on next page...)*

(continued...)

| Method   | Returns | Arguments | Description                     |
|----------|---------|-----------|---------------------------------|
| size     | size_t  | —         | Delegate to base-class size     |
| is_empty | bit     | —         | Delegate to base-class is_empty |

## 4.11 permute\_iterator\_base #(T, P)

**Kind:** class

Base for permutation iterators; manages the permutation vector and index arithmetic

**Constructor:** new(v)

| Method                 | Returns   | Arguments | Description                                        |
|------------------------|-----------|-----------|----------------------------------------------------|
| bind_vector            | void      | v         | Bind a vector and initialize the permutation state |
| size                   | size_t    | —         | Return the size of the bound vector                |
| is_empty               | bit       | —         | Return 1 if unbound or empty                       |
| set                    | void      | t         | Not implemented for permutation iterators          |
| get                    | T         | —         | Not implemented for permutation iterators          |
| get_nth                | T         | n         | Return the nth element in the current permutation  |
| get_permutation_index  | longint   | —         | Return the current permutation index (pix)         |
| get_permutation_vector | vector#() | —         | Return a copy of the current permutation vector    |
| skip                   | bit       | distance  | Move pix forward or backward by distance           |
| print                  | void      | —         | Print the current permutation to stdout            |

## 4.12 permute\_random\_iterator #(T, P)

**Kind:** class

**Extends:** permute\_iterator\_base #(T, P)

**Implements:** random\_intf #(T, P)

Jumps to a randomly chosen permutation of the bound vector

**Constructor:** new(vec)

| Method           | Returns | Arguments | Description                           |
|------------------|---------|-----------|---------------------------------------|
| size             | size_t  | —         | Delegate to base-class size           |
| is_empty         | bit     | —         | Delegate to base-class is_empty       |
| set_seed         | void    | seed      | Seed the RNG                          |
| set_default_seed | void    | —         | Reset RNG to the default seed         |
| random           | bit     | —         | Jump to a randomly chosen permutation |
| skip             | bit     | distance  | Delegate to base-class skip           |

### 4.13 permute\_iterator #(T, P)

**Kind:** class

**Extends:** permute\_iterator\_base #(T, P)

**Implements:** fwd\_intf #(T, P), bkwd\_intf #(T, P)

Traverses all permutations of a bound vector in lexicographic order, forward or backward

**Constructor:** new(vec)

| Method       | Returns | Arguments | Description                                   |
|--------------|---------|-----------|-----------------------------------------------|
| size         | size_t  | —         | Delegate to base-class size                   |
| is_empty     | bit     | —         | Delegate to base-class is_empty               |
| set          | void    | t         | Not implemented (satisfies interface)         |
| get          | T       | —         | Not implemented (satisfies interface)         |
| first        | bit     | —         | Reset to permutation 0                        |
| next         | bit     | —         | Advance to the next lexicographic permutation |
| is_last      | bit     | —         | Return 1 if at the last permutation           |
| at_end       | bit     | —         | Return 1 if past the last permutation         |
| last         | bit     | —         | Jump to the last permutation                  |
| prev         | bit     | —         | Retreat to the previous permutation           |
| is_first     | bit     | —         | Return 1 if at permutation 0                  |
| at_beginning | bit     | —         | Return 1 if before permutation 0              |
| skip         | bit     | distance  | Delegate to base-class skip                   |

### 4.14 range\_base

**Kind:** virtual class

Stores validated upper and lower bound indices for a range slice

**Constructor:** new(size, lower\_bound, upper\_bound)

| Method          | Returns | Arguments | Description                  |
|-----------------|---------|-----------|------------------------------|
| get_lower_bound | index_t | —         | Return the lower bound index |
| get_upper_bound | index_t | —         | Return the upper bound index |

## 4.15 range #(T, P)

**Kind:** class

**Extends:** range\_base

**Implements:** fwd\_intf #(T, P), bkwd\_intf #(T, P)

Wraps a bidirectional iterator to expose only a sub-range [lower\_bound, upper\_bound]

**Constructor:** new(it, lower\_bound, upper\_bound)

| Method       | Returns | Arguments | Description                                                 |
|--------------|---------|-----------|-------------------------------------------------------------|
| set          | void    | t         | Set the value at the current index via the wrapped iterator |
| get          | T       | —         | Get the value at the current index via the wrapped iterator |
| size         | size_t  | —         | Return ub - lb + 1                                          |
| is_empty     | bit     | —         | Return 1 if the wrapped forward iterator is null or empty   |
| first        | bit     | —         | Move to the lower-bound position                            |
| next         | bit     | —         | Advance within the range                                    |
| is_last      | bit     | —         | Return 1 if at the upper-bound position                     |
| at_end       | bit     | —         | Return 1 if past the upper bound                            |
| last         | bit     | —         | Move to the upper-bound position                            |
| prev         | bit     | —         | Retreat within the range                                    |
| is_first     | bit     | —         | Return 1 if at the lower-bound position                     |
| at_beginning | bit     | —         | Return 1 if before the lower bound                          |
| skip         | bit     | distance  | Delegate skip to the wrapped iterator                       |

## 4.16 map\_range #(KEY, T, P)

**Kind:** class

**Extends:** range #(T, P)

A range slice over a map; adds get\_index() to expose the current map key

**Constructor:** new(it, lower\_bound, upper\_bound)

| Method                 | Returns | Arguments | Description                                         |
|------------------------|---------|-----------|-----------------------------------------------------|
| <code>get_index</code> | KEY     | —         | Return the map key at the current iterator position |

# Algorithms

## 5.1 algo #(T, P)

**Kind:** class

Collection of static algorithm functions that operate on forward iterators and predicates

| Method   | Returns  | Arguments | Description                                                    |
|----------|----------|-----------|----------------------------------------------------------------|
| count    | uint32_t | iter, p   | Count elements for which predicate p is true (static)          |
| all_of   | bit      | iter, p   | Return 1 if p is true for all elements (static)                |
| none_of  | bit      | iter, p   | Return 1 if p is false for all elements (static)               |
| any_of   | bit      | iter, p   | Return 1 if p is true for at least one element (static)        |
| min      | T        | iter      | Return the minimum element according to P::compare (static)    |
| max      | T        | iter      | Return the maximum element according to P::compare (static)    |
| find     | void     | iter, p   | Advance iter to the first element for which p is true (static) |
| for_each | void     | iter, fn  | Apply fn.f() to every element in the iteration (static)        |

## 5.2 accum #(T, P, A)

**Kind:** class

Accumulation algorithm; applies an accumulator function to all elements of an iteration

| Method     | Returns | Arguments     | Description                                                |
|------------|---------|---------------|------------------------------------------------------------|
| accumulate | void    | iter, fn, acc | Apply fn.f(t, acc) to every element, updating acc (static) |

### 5.3 fcn\_base

**Kind:** virtual class

Non-parameterized root base for function-object classes

*No public methods.*

### 5.4 fcn #(T)

**Kind:** virtual class

**Extends:** fcn\_base

Abstract single-argument function object; implement f() to define a per-element operation

| Method | Returns | Arguments | Description                                 |
|--------|---------|-----------|---------------------------------------------|
| f      | void    | t         | Apply the function to item t (pure virtual) |

### 5.5 accum\_fcn #(T, A)

**Kind:** virtual class

**Extends:** fcn\_base

Abstract accumulator function object; implement f() to fold an item into an accumulator

| Method | Returns | Arguments | Description                                                                       |
|--------|---------|-----------|-----------------------------------------------------------------------------------|
| f      | void    | t, a      | Apply the accumulation function: update accumulator a using item t (pure virtual) |

### 5.6 predicate\_base

**Kind:** virtual class

Non-parameterized root base for predicate classes

*No public methods.*

### 5.7 predicate #(T)

**Kind:** virtual class

**Extends:** predicate\_base

Abstract predicate; implement is\_true() to define a boolean test on elements of type T

| Method                | Returns | Arguments      | Description                                                       |
|-----------------------|---------|----------------|-------------------------------------------------------------------|
| <code>is_true</code>  | bit     | <code>t</code> | Return 1 if the predicate holds for <code>t</code> (pure virtual) |
| <code>is_false</code> | bit     | <code>t</code> | Return <code>!is_true(t)</code>                                   |

## 5.8 `and_pred #(T)`

**Kind:** class

**Extends:** `predicate #(T)`

Composite predicate that is true only when both sub-predicates are true

**Constructor:** `new(_a, _b)`

| Method               | Returns | Arguments      | Description                                              |
|----------------------|---------|----------------|----------------------------------------------------------|
| <code>is_true</code> | bit     | <code>t</code> | Return <code>a.is_true(t) &amp;&amp; b.is_true(t)</code> |

## 5.9 `or_pred #(T)`

**Kind:** class

**Extends:** `predicate #(T)`

Composite predicate that is true when either sub-predicate is true

**Constructor:** `new(_a, _b)`

| Method               | Returns | Arguments      | Description                      |
|----------------------|---------|----------------|----------------------------------|
| <code>is_true</code> | bit     | <code>t</code> | Return <code>a.is_true(t)</code> |

## 5.10 `not_pred #(T)`

**Kind:** class

**Extends:** `predicate #(T)`

Negating predicate; `is_true` returns the inverse of the wrapped predicate

**Constructor:** `new(_p)`

| Method               | Returns | Arguments      | Description                       |
|----------------------|---------|----------------|-----------------------------------|
| <code>is_true</code> | bit     | <code>t</code> | Return <code>p.is_false(t)</code> |

## 5.11 `always_true #(T)`

**Kind:** class

**Extends:** `predicate #(T)`

Constant predicate that is always true regardless of the element value

| Method               | Returns          | Arguments      | Description      |
|----------------------|------------------|----------------|------------------|
| <code>is_true</code> | <code>bit</code> | <code>t</code> | Always returns 1 |

## 5.12 `always_false` $\#(T)$

**Kind:** class

**Extends:** `predicate`  $\#(T)$

Constant predicate that is always false regardless of the element value

| Method               | Returns          | Arguments      | Description      |
|----------------------|------------------|----------------|------------------|
| <code>is_true</code> | <code>bit</code> | <code>t</code> | Always returns 0 |

# Linked Structures

## 6.1 node

**Kind:** class

**Extends:** object

Named, markable object; provides naming service and a mark bit for graph traversals

**Constructor:** new(nm)

| Method    | Returns | Arguments | Description                              |
|-----------|---------|-----------|------------------------------------------|
| set_name  | void    | nm        | Set the node's name                      |
| get_name  | string  | —         | Return the node's name                   |
| mark      | void    | —         | Set the mark bit to 1                    |
| unmark    | void    | —         | Clear the mark bit to 0                  |
| get_mark  | bit     | —         | Return the current value of the mark bit |
| is_marked | (bit)   | —         | Return the mark bit (alias for get_mark) |

## 6.2 tree

**Kind:** class

**Extends:** node

Classic named tree; each node has one parent and zero or more named children; supports hierarchical path lookup

**Constructor:** new(nm, p)

| Method        | Returns | Arguments | Description                                          |
|---------------|---------|-----------|------------------------------------------------------|
| insert        | void    | t         | Insert tree node t as a child of this node           |
| get_parent    | tree    | —         | Return a handle to this node's parent                |
| get_full_name | string  | —         | Return the full dot-separated hierarchical path name |

*(continued on next page...)*

(continued...)

| Method                    | Returns               | Arguments         | Description                                                             |
|---------------------------|-----------------------|-------------------|-------------------------------------------------------------------------|
| <code>first_child</code>  | <code>bit</code>      | <code>nm</code>   | Set <code>nm</code> to the name of the first child; return 1 on success |
| <code>next_child</code>   | <code>bit</code>      | <code>nm</code>   | Set <code>nm</code> to the name of the next child; return 1 on success  |
| <code>num_children</code> | <code>size_t</code>   | —                 | Return the number of direct children                                    |
| <code>size</code>         | <code>size_t</code>   | —                 | Return the number of children (via deque)                               |
| <code>get_children</code> | <code>deque#()</code> | —                 | Return a deque containing all direct children                           |
| <code>get_child</code>    | <code>tree</code>     | <code>nm</code>   | Return the child with the given name, or null                           |
| <code>find</code>         | <code>tree</code>     | <code>path</code> | Find a descendant by dot-separated hierarchical path                    |
| <code>mark_all</code>     | <code>void</code>     | —                 | Recursively mark this node and all descendants                          |
| <code>unmark_all</code>   | <code>void</code>     | —                 | Recursively unmark this node and all descendants                        |

## 6.3 `tree_iterator`

**Kind:** class

**Implements:** `fwd_intf #(tree, class_traits#(tree))`

Pre-order depth-first iterator over a tree; uses an internal stack for traversal

**Constructor:** `new(t)`

| Method                | Returns             | Arguments      | Description                                                     |
|-----------------------|---------------------|----------------|-----------------------------------------------------------------|
| <code>size</code>     | <code>size_t</code> | —              | Count all nodes in the tree (has side effect of changing marks) |
| <code>is_empty</code> | <code>bit</code>    | —              | Return 1 if no tree is bound                                    |
| <code>set</code>      | <code>void</code>   | <code>t</code> | Not implemented (satisfies interface)                           |
| <code>get</code>      | <code>tree</code>   | —              | Return the tree node at the top of the traversal stack          |
| <code>first</code>    | <code>bit</code>    | —              | Reset the traversal; push the root node                         |
| <code>next</code>     | <code>bit</code>    | —              | Advance to the next node (pop current, push its children)       |
| <code>is_last</code>  | <code>bit</code>    | —              | Return 1 if only one node remains on the stack                  |
| <code>at_end</code>   | <code>bit</code>    | —              | Return 1 if the traversal stack is empty                        |

(continued on next page...)

*(continued...)*

| Method | Returns | Arguments | Description                                          |
|--------|---------|-----------|------------------------------------------------------|
| skip   | bit     | distance  | Skip forward by distance nodes; cannot skip backward |